

# Projeto Threads - Priority

## Participantes:

Pedro Arthur Santana Patriota

Yasmim Vitoria Silva de Oliveira

Pedro Augusto de Almada Lobo Fonseca

Arthur Brito Medeiros

## 1. Decisões de implementação (*design*)

### Estrutura de Dados

pintos/src/threads/thread.h

```
1 struct thread
2 {
3     /* Owned by thread.c. */
4     tid_t tid; /* Thread identifier. */
5     enum thread_status status; /* Thread state. */
6     char name[16]; /* Name (for debugging purposes). */
7     uint8_t *stack; /* Saved stack pointer. */
8     int priority;
9     int base_priority; /*guarda a prioridade real da thread e direciona a thread*/
10    struct list locks_held; /*lista de todos os locks que a thread segura no momento e percorrida pra saber se algum lock tem waiter de prioridade*/
11    struct lock *waiting_on_lock; /*ponteiro para lock que a thread está esperando*/
12    struct list_elem allelem; /* List element for all threads list. */
13
14    int64_t wakeup_tick;
```

- **priority** e **base\_priority**: Diferenciam prioridade efetiva (doad) da original, permitindo restaurar após liberação do lock.
- **locks\_held**: Rastreia todos os *locks* que a thread segura, necessário para recalcular prioridade ao liberar um lock.
- **waiting\_on\_lock**: Registra qual *lock* a thread aguarda, permitindo propagar doação em cadeia através de dependências aninhadas.

pintos/src/threads/synch.h

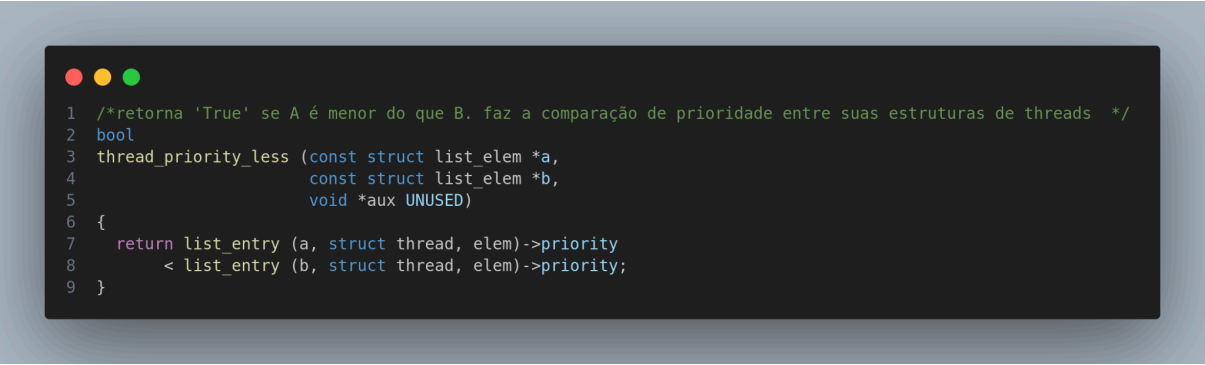
```
1 /* Lock. */
2 struct lock
3 {
4     struct thread *holder; /* Thread holding lock (for debugging). */
5     struct semaphore semaphore; /* Binary semaphore controlling access. */
6     struct list_elem elem /*para que cada lock possa ser um nó na lista 'locks_held' de sua thread*/
7 };
8
```

- **elem**: Utilizado como estrutura para servir como um nó na lista de elementos que possuem lock.

Algoritmos

pintos/threads/threads.c

### **Funções Criadas**



```
1  /*retorna 'True' se A é menor do que B. faz a comparação de prioridade entre suas estruturas de threads */
2  bool
3  thread_priority_less (const struct list_elem *a,
4                       const struct list_elem *b,
5                       void *aux UNUSED)
6  {
7      return list_entry (a, struct thread, elem)->priority
8          < list_entry (b, struct thread, elem)->priority;
9  }
```

***thread\_priority\_less***: Utilizada para análise de prioridades entre elementos, como auxiliar de tomada de decisão

```

1 void
2 thread_recalculate_priority (struct thread *t)
3 {
4     int priority = t->base_priority;
5     struct list_elem *e;
6
7     //percorre cada lock que a thread ainda segura
8     for (e = list_begin (&t->locks_held);
9          e != list_end (&t->locks_held);
10          e = list_next (e))
11     {
12         struct lock *l = list_entry (e, struct lock, elem);
13         if (!list_empty (&l->semaphore.waiters))
14         {
15             /* encontra o waiter de maior prioridade neste lock*/
16             int waiter_pri =
17                 list_entry (list_max (&l->semaphore.waiters,
18                                     thread_priority_less, NULL),
19                             struct thread, elem)->priority;
20             if (waiter_pri > priority)
21                 priority = waiter_pri; // mantém a doação se necessário
22         }
23     }
24
25     t->priority = priority;
26 }

```

***thread\_recalculate\_priority:*** A lógica é recalcular a prioridade efetiva entre a máxima prioridade de base e a maior prioridade de qualquer thread esperando algum lock que está sendo segurado. Caso contrário, segura mais nenhum lock com espera e volta ao base\_priority

```

1 void thread_yield_if_needed (void)
2 {
3     if (list_empty(&ready_list))
4         return;
5
6     struct thread *next = list_entry(list_max (&ready_list, thread_priority_less, NULL), struct thread, elem);
7
8     if(next->priority > thread_current ()->priority)
9         thread_yield (); // cede cpu para quem tem maior prioridade
10 }

```

***thread\_yield\_if\_needed:*** Otimização da função thread\_yield. Na função original, uma thread que acabou de liberar um lock de alta prioridade continuaria rodando mesmo com uma thread de prioridade maior esperando na fila, ou seja, ficaria em loop sem existir a liberação da cpu

## Funções modificadas

```
1 static void
2 init_thread (struct thread *t, const char *name, int priority)
3 {
4     enum intr_level old_level;
5
6     ASSERT (t != NULL);
7     ASSERT (PRI_MIN <= priority && priority <= PRI_MAX);
8     ASSERT (name != NULL);
9
10    memset (t, 0, sizeof *t);
11    t->status = THREAD_BLOCKED;
12    strcpy (t->name, name, sizeof t->name);
13    t->stack = (uint8_t *) t + PGSIZE;
14    t->priority = priority;
15    t->base_priority = priority;           // base de início
16    list_init (&t->locks_held);          // lista de locks vazias
17    t->waiting_on_lock = NULL;           // setado como NULL: não espera nenhum lock
18    t->magic = THREAD_MAGIC;
19
20    old_level = intr_disable ();
21    list_push_back (&all_list, &t->allelem);
22    intr_set_level (old_level);
23 }
```

Criação de listas vazias de locks, setando variáveis

```
1 static struct thread *next_thread_to_run (void)
2 {
3     if (list_empty (&ready_list))
4         return idle_thread;
5     // pega o de maior prioridade, onde quer que esteja na lista
6     // utilizar list_max por conta da mudança de prioridade causada por prioridades de threads que já estão na ready_list
7     struct list_elem *max_elem = list_max (&ready_list, thread_priority_less, NULL);
8     list_remove (max_elem);
9     return list_entry (max_elem, struct thread, elem);
10 }
11
```

Retirada de thread de maior prioridade da lista de wait, para execução

```

1 void thread_set_priority (int new_priority)
2 {
3     // altera base, mantém doação se ativa
4     struct thread *cur = thread_current ();
5     cur->base_priority = new_priority;
6     thread_recalculate_priority (cur);
7     thread_yield_if_needed ();
8 }
9

```

```

1 void lock_acquire (struct lock *lock)
2 {
3     ASSERT (lock != NULL);
4     ASSERT (!intr_context ());
5     ASSERT (!lock_held_by_current_thread (lock));
6
7     struct thread *cur = thread_current ();
8
9     // doa prioridade antes de bloquear, em cadeia
10
11     if (lock->holder != NULL)
12     {
13         // registra a espera
14         cur->waiting_on_lock = lock;
15
16         /* sobe as prioridades por nível considerando a cadeia de dependência de lock */
17         struct thread *holder = lock->holder;
18         while (holder != NULL && holder->priority < cur->priority)
19         {
20             holder->priority = cur->priority; // doa
21
22             if (holder->waiting_on_lock == NULL)
23                 break;
24             holder = holder->waiting_on_lock->holder; // sobe o nível da cadeia
25         }
26     }
27
28     sema_down (&lock->semaphore);
29
30     cur->waiting_on_lock = NULL;
31     lock->holder = cur;
32     list_push_back (&cur->locks_held, &lock->elem);
33 }
34

```

A função retificada implementa doação de prioridade: quando uma thread de menor prioridade segura o lock, a thread atual transfere sua prioridade para ela, inclusive propagando essa doação em cadeias de bloqueios aninhados, para acelerar a liberação do recurso. O while é o que implementa o Exemplo 2 do escopo: Thread3 (60) espera lockB ->

doa 60 para Thread2 -> Thread2 espera lockA -> doa 60 para Thread1. Tudo em uma única passagem pelo loop.

pintos/threads/synch.c

### **Funções Criadas**

```
1 static bool
2 semaphore_elem_priority_less (const struct list_elem *a,
3                               const struct list_elem *b,
4                               void *aux UNUSED)
5 {
6     struct semaphore_elem *sa =
7         list_entry (a, struct semaphore_elem, elem);
8     struct semaphore_elem *sb =
9         list_entry (b, struct semaphore_elem, elem);
10
11     struct list_elem *ma = list_max (&sa->semaphore.waiters,
12                                     thread_priority_less, NULL);
13     struct list_elem *mb = list_max (&sb->semaphore.waiters,
14                                     thread_priority_less, NULL);
15
16     int pa = list_entry (ma, struct thread, elem)->priority;
17     int pb = list_entry (mb, struct thread, elem)->priority;
18
19     return pa < pb;
20 }
```

Função que compara dois semaphore\_elem com base na thread de maior prioridade que está aguardando em seus respectivos semáforos. Usada para selecionar o elemento com a maior prioridade na lista de espera de uma variável de condição.

### **Funções Modificadas**

```

1 void
2 sema_up (struct semaphore *sema)
3 {
4     enum intr_level old_level;
5
6     ASSERT (sema != NULL);
7
8     old_level = intr_disable ();
9     if (!list_empty (&sema->waiters))
10    {
11        struct list_elem *max_elem =
12            list_max (&sema->waiters, thread_priority_less, NULL);
13        list_remove (max_elem);
14        thread_unblock (list_entry (max_elem, struct thread, elem));
15    }
16    sema->value++;
17    intr_set_level (old_level);
18
19    if (!intr_context ())
20        thread_yield_if_needed ();
21 }

```

A função foi modificada para sempre acordar a thread de maior prioridade entre as que estão esperando no semáforo, utilizando `list_max` para garantir correção mesmo com mudanças dinâmicas de prioridade (como priority donation). Além disso, após desbloquear essa thread, a thread atual verifica se deve ceder a CPU, fazendo um yield caso exista uma thread pronta com prioridade maior (via `thread_yield_if_needed`), garantindo que ela execute imediatamente.

```

1 void
2 cond_signal (struct condition *cond, struct lock *lock UNUSED)
3 {
4     ASSERT (cond != NULL);
5     ASSERT (lock != NULL);
6     ASSERT (!intr_context ());
7     ASSERT (lock_held_by_current_thread (lock));
8
9     if (!list_empty (&cond->waiters))
10    {
11        struct list_elem *max_elem =
12            list_max (&cond->waiters, semaphore_elem_priority_less, NULL);
13        list_remove (max_elem);
14        sema_up (&list_entry (max_elem, struct semaphore_elem, elem)->semaphore);
15    }
16 }

```

Ao invés de acordar a primeira das threads que estão esperando na variável de condição, a função agora escolhe aquela com maior prioridade. Para isso, percorre a lista de waiters e identifica o elemento cujo semáforo está associado à thread de maior prioridade (via `list_max`, usando `semaphore_elem_priority_less` como função de comparação). Esse elemento é então removido da lista, e seu semáforo é incrementado (via `sema_up`), acordando a thread.

## Sincronização

### 2. Testes

**\*\*Todos os testes de mlfqs falharam**

**\*\* Todos os testes de priority passaram**



```

pintos -v -k -T 60 --qemu -- -q run priority-change < /dev/null 2> tests/threads/priority-change.errors > tests/threads/priority-change.output
perl -I../.. ../tests/threads/priority-change.ck tests/threads/priority-change tests/threads/priority-change.result
pass tests/threads/priority-change
pintos -v -k -T 60 --qemu -- -q run priority-donate-one < /dev/null 2> tests/threads/priority-donate-one.errors > tests/threads/priority-donate-one.output
perl -I../.. ../tests/threads/priority-donate-one.ck tests/threads/priority-donate-one tests/threads/priority-donate-one.result
pass tests/threads/priority-donate-one
pintos -v -k -T 60 --qemu -- -q run priority-donate-multiple < /dev/null 2> tests/threads/priority-donate-multiple.errors > tests/threads/priority-donate-multiple.output
perl -I../.. ../tests/threads/priority-donate-multiple.ck tests/threads/priority-donate-multiple tests/threads/priority-donate-multiple.result
pass tests/threads/priority-donate-multiple
pintos -v -k -T 60 --qemu -- -q run priority-donate-multiple2 < /dev/null 2> tests/threads/priority-donate-multiple2.errors > tests/threads/priority-donate-multiple2.output
perl -I../.. ../tests/threads/priority-donate-multiple2.ck tests/threads/priority-donate-multiple2 tests/threads/priority-donate-multiple2.result
pass tests/threads/priority-donate-multiple2
pintos -v -k -T 60 --qemu -- -q run priority-donate-nest < /dev/null 2> tests/threads/priority-donate-nest.errors > tests/threads/priority-donate-nest.output
perl -I../.. ../tests/threads/priority-donate-nest.ck tests/threads/priority-donate-nest tests/threads/priority-donate-nest.result
pass tests/threads/priority-donate-nest
pintos -v -k -T 60 --qemu -- -q run priority-donate-sema < /dev/null 2> tests/threads/priority-donate-sema.errors > tests/threads/priority-donate-sema.output
perl -I../.. ../tests/threads/priority-donate-sema.ck tests/threads/priority-donate-sema tests/threads/priority-donate-sema.result
pass tests/threads/priority-donate-sema
pintos -v -k -T 60 --qemu -- -q run priority-donate-lower < /dev/null 2> tests/threads/priority-donate-lower.errors > tests/threads/priority-donate-lower.output
perl -I../.. ../tests/threads/priority-donate-lower.ck tests/threads/priority-donate-lower tests/threads/priority-donate-lower.result
pass tests/threads/priority-donate-lower
pintos -v -k -T 60 --qemu -- -q run priority-fifo < /dev/null 2> tests/threads/priority-fifo.errors > tests/threads/priority-fifo.output
perl -I../.. ../tests/threads/priority-fifo.ck tests/threads/priority-fifo tests/threads/priority-fifo.result
pass tests/threads/priority-fifo
pintos -v -k -T 60 --qemu -- -q run priority-preempt < /dev/null 2> tests/threads/priority-preempt.errors > tests/threads/priority-preempt.output
perl -I../.. ../tests/threads/priority-preempt.ck tests/threads/priority-preempt tests/threads/priority-preempt.result
pass tests/threads/priority-preempt
pintos -v -k -T 60 --qemu -- -q run priority-sema < /dev/null 2> tests/threads/priority-sema.errors > tests/threads/priority-sema.output
perl -I../.. ../tests/threads/priority-sema.ck tests/threads/priority-sema tests/threads/priority-sema.result
pass tests/threads/priority-sema
pintos -v -k -T 60 --qemu -- -q run priority-condvar < /dev/null 2> tests/threads/priority-condvar.errors > tests/threads/priority-condvar.output
perl -I../.. ../tests/threads/priority-condvar.ck tests/threads/priority-condvar tests/threads/priority-condvar.result
pass tests/threads/priority-condvar
pintos -v -k -T 60 --qemu -- -q run priority-donate-chain < /dev/null 2> tests/threads/priority-donate-chain.errors > tests/threads/priority-donate-chain.output
perl -I../.. ../tests/threads/priority-donate-chain.ck tests/threads/priority-donate-chain tests/threads/priority-donate-chain.result
pass tests/threads/priority-donate-chain

```

### 3. Arquivos alterados ou adicionados

threads.c

thread.h

synch.c

synch.h